

Dossier de déploiement — Écosystème ESIG (pour M. Kalipé)

Écosystème de **8 sites statiques** (HTML/CSS/JS, aucun framework) + **2 services Node** (assistant NOVA et traitement des formulaires). **206 pages** au total (dont 145 fiches formation). Ce document est le mode d'emploi complet de la mise en ligne, de la préparation des livrables aux tests finaux.

Version du dossier : 11/08/2026 — réorganisation du portail en **8 espaces** : « International » devient **Coopération & Relations internationales** (`cooperation.esig.tg`), « Entreprises » est absorbé par **Carrières & Insertion** (`carrieres.esig.tg`), et un espace **Alumni** (`alumni.esig.tg`) est créé (cf. CHANGELOG v2.2.0). Fichiers de référence en **§16 (Annexe)**.

o. Pré-requis

- **Accès serveurs** : accès SSH au serveur de déploiement (**B**) et compte administrateur sur le Nginx Proxy Manager (**A**).
- **DNS** : les **8 sous-domaines** doivent pointer vers le NPM (serveur A) — `esig.tg` , `admission.` , `executive.` , `cooperation.` , `carrieres.` , `alumni.` , `news.` , `tech.esig.tg` . Prévoir la redirection des anciens `international.esig.tg` et `entreprises.esig.tg` (§12), et un sous-domaine **privé** pour l'administration de contenu (ex. `admin.esig.tg` , §10).
- **Poste de préparation** : Node ≥ 18 (pour générer les livrables et lancer les services).
- **Clé API du modèle** (pour NOVA) : à détenir **côté serveur uniquement** (jamais dans le code).
- **Compte SMTP** (pour l'envoi des formulaires par e-mail) : hôte, port, identifiant, mot de passe.

1. Architecture de déploiement (importante)

L'application est publiée **derrière un reverse proxy Nginx Proxy Manager (NPM)** déjà en place sur un **serveur distinct**, relié au serveur de déploiement par une **interface locale (réseau privé)**.

```
Internet -HTTPS-> [ Serveur A : Nginx Proxy Manager ] -interface locale (HTTP)-> [ Serveur B : déploiement ]
```

• termine le SSL (certificats)	• sert
les 8 sites statiques (Nginx, SANS SSL)	
• route chaque sous-domaine vers B	• héberge
le relais NOVA (Node, 127.0.0.1:8787)	
• pose HSTS + en-têtes (option)	• héberge
le service formulaires (Node, 127.0.0.1:8788)	

Conséquences à respecter : - Le serveur B **ne fait PAS de SSL** : pas de certbot, et **surtout aucun Caddy** (qui générerait du TLS en doublon). Le SSL est **entièrement géré par NPM** (serveur A). - Le serveur B sert les fichiers via un **Nginx simple, en HTTP**, sur l'interface locale ; NPM lui transmet les requêtes en conservant l'en-tête `Host`. - **Routes API (NOVA + formulaires) : same-origin par sous-domaine**. Le navigateur appelle `/api/assistant` et `/api/formulaire` (chemins **relatifs**) sur son propre sous-domaine ; le Nginx du serveur B proxifie ces chemins vers les services locaux. Aucune requête cross-origin → aucune dépendance au CORS (voir §7 et §8).

2. Vue d'ensemble des sites

Sous-domaine	Dossier source	Contenu	Pages
esig.tg (www)	sites/www	Institutionnel (gouvernance, parcours académique, pages légales, demande de document)	16
admission.esig.tg	sites/admission	Commercial + 66 fiches formation académique	71
executive.esig.tg	sites/executive	Formation continue + 79 fiches	88
cooperation.esig.tg	sites/cooperation	Coopération & Relations internationales (← International)	6
carrieres.esig.tg	sites/carrieres	Carrières & Insertion professionnelle (← Entreprises, élargi)	5
alumni.esig.tg	sites/alumni	Réseau des diplômés (nouveau)	5
news.esig.tg	sites/news	Média « ESIG News » + Agenda	3
tech.esig.tg	sites/tech	ESIG TECH — innovations, projets étudiants, labos, partenaires	12
		Total	206

Les **pages légales** (mentions, confidentialité, cookies, accessibilité) sont hébergées **une seule fois sur esig.tg** ; les autres sites y renvoient en absolu. À prévoir plus tard : un sous-domaine **privé** d'administration de contenu (§10).

3. Générer les livrables (poste de préparation, Node ≥ 18)

Depuis la racine du projet, dans l'ordre :

```
node build/convertir-formations.js      # data/formations.json (145 formations)
node build/generer-fiches.js admission # 66 fiches académiques
node build/generer-fiches.js executive # 79 fiches continues
node build/build.js                    # assemble toutes les pages (.src.html -> .html)
node build/audit.js                    # contrôle structure + accessibilité (doit être
206/206)
node build/assembler.js                 # crée dist/<site>/ AUTONOMES (à déployer)
node build/verifier-liens.js            # vérifie les liens/ressources internes (doit être
0 cassé)
```

Chaque `dist/<site>/` contient déjà `shared/`, `data/`, `medias/`, `robots.txt`, `sitemap.xml` : **il se copie tel quel à la racine web du sous-domaine**. Les dossiers internes (`medias/_*`) et les gabarits de build (`shared/components/*.html`) sont **automatiquement exclus** du paquet (voir §9).

3.1 — Régler l'assistant NOVA avant déploiement

Les pages pointent, en développement, vers `http://localhost:8787/api/assistant` . **Avant de déployer**, remplacez cette valeur dans `dist/` :

```
# Pour ACTIVER NOVA (relais déployé, voir §7) – endpoint RELATIF, same-origin par sous-
domaine :
find dist -name "*.html" -exec sed -i
's|http://localhost:8787/api/assistant|/api/assistant|g' {} +

# Ou pour le GARDER MASQUÉ tant que le relais n'est pas prêt (endpoint vide) :
find dist -name "*.html" -exec sed -i 's|http://localhost:8787/api/assistant||g' {} +
```

(Endpoint `/api/assistant` = appelé sur le sous-domaine courant, proxifié vers le relais par le Nginx du serveur B, voir §7. Endpoint vide = widget invisible, sans erreur.)

3.2 — Régler le service des formulaires avant déploiement

Même logique pour les formulaires (§8) : en développement ils pointent vers `http://localhost:8788/api/formulaire` . Passez l'endpoint en **relatif** (same-origin) :

```
find dist -name "*.html" -exec sed -i
's|http://localhost:8788/api/formulaire|/api/formulaire|g' {} +
```

Ces deux `sed` couvrent **tous** les sous-domaines, y compris `tech` (widget NOVA + formulaire « Proposer un projet »).

4. Déploiement des fichiers

Pour chaque site, copier le contenu de `dist/<site>/` vers la racine du sous-domaine, par ex. :

```
rsync -av --delete dist/www/          deploy@serveur:/var/www/esig.tg/
rsync -av --delete dist/admission/    deploy@serveur:/var/www/admission.esig.tg/
rsync -av --delete dist/executive/    deploy@serveur:/var/www/executive.esig.tg/
rsync -av --delete dist/cooperation/  deploy@serveur:/var/www/cooperation.esig.tg/
rsync -av --delete dist/carrieres/    deploy@serveur:/var/www/carrieres.esig.tg/
rsync -av --delete dist/alumni/       deploy@serveur:/var/www/alumni.esig.tg/
rsync -av --delete dist/news/         deploy@serveur:/var/www/news.esig.tg/
rsync -av --delete dist/tech/         deploy@serveur:/var/www/tech.esig.tg/
```

`--delete` supprime côté serveur les fichiers absents du paquet : la racine du sous-domaine doit donc ne contenir **que** le site (pas de fichier étranger).

5. Serveur B — Nginx statique (SANS SSL, derrière NPM)

Le serveur de déploiement fait tourner **un Nginx simple**, en **HTTP** sur l'interface locale, qui sert les 8 sites différenciés par l'en-tête `Host` (transmis par NPM). **Aucun bloc SSL ici.**

Bloc type (à décliner pour les **8 sous-domaines** en changeant `server_name` et `root`) :

```

server {
    listen 80;                                # interface locale ; NPM proxifie vers ce port
    server_name esig.tg;                      # ← adapter par sous-domaine (Host transmis par
NPM)
    root /var/www/esig.tg;                   # ← adapter par sous-domaine
    index index.html;

    # Relais NOVA – MÊME ORIGINE : /api/assistant -> relais Node local (voir §7)
    location = /api/assistant {
        proxy_pass http://127.0.0.1:8787;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
    }

    # Service formulaires – MÊME ORIGINE : /api/formulaire -> service Node local (voir §8)
    location = /api/formulaire {
        proxy_pass http://127.0.0.1:8788;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
    }

    # En-têtes de sécurité (voir §11) – OU à poser dans NPM (onglet « Advanced »), sans
    doublon
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "DENY" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
    add_header Permissions-Policy "geolocation=(), microphone=(), camera=()" always;
    add_header Content-Security-Policy "default-src 'self'; img-src 'self' data:; script-src
'self' 'unsafe-inline' https://static.cloudflareinsights.com; style-src 'self' 'unsafe-
inline'; font-src 'self'; connect-src 'self' https://cloudflareinsights.com; form-action
'self' https:; frame-ancestors 'none'; base-uri 'self'" always;

    # Cache des actifs
    location ~* \.(css|js|png|jpg|jpeg|webp|svg|ico|woff2?)$ { expires 30d; add_header Cache-
Control "public"; }
    location = /sitemap.xml { expires 1d; }
    location = /robots.txt { expires 1d; }

    gzip on; gzip_types text/css application/javascript application/json image/svg+xml;

```

```
error_page 404 /404.html;    # (le www embarque une 404 ; sinon /index.html)
}
```

Côté NPM (serveur A), pour chaque sous-domaine : créer un *Proxy Host* pointant vers l'IP locale du serveur B (port 80), **Host conservé**, SSL activé (§6), *Block Common Exploits* activé. NPM transmet **tout** (pages + `/api/assistant` + `/api/formulaire`) à Nginx B, qui route les API vers les services localement. **Les nouveaux sous-domaines** `cooperation.esig.tg`, `carrieres.esig.tg`, `alumni.esig.tg` et `tech.esig.tg` **suivent le même modèle** : même bloc, en adaptant `server_name` et `root`.

Note CSP. `connect-src 'self'` suffit : NOVA et le service formulaires (§8) sont appelés en **même origine**. Les vignettes vidéo des pages « Émissions » sont des **images locales** ; le clic ouvre YouTube en **nouvel onglet** (navigation, pas d'iframe) → `img-src 'self'` et `frame-ancestors 'none'` restent valables. `script-src 'unsafe-inline'` reste requis pour les scripts inline (config, validation de formulaires) — externalisables plus tard pour durcir la CSP.

6. SSL — géré par Nginx Proxy Manager (serveur A)

Aucune configuration SSL sur le serveur de déploiement (B). Tout se fait dans NPM : - Pour chaque *Proxy Host*, onglet **SSL** → « Request a new SSL Certificate » (Let's Encrypt), puis activer **Force SSL**, **HTTP/2 Support** et **HSTS Enabled**. - Renouvellement automatique assuré par NPM. - Un certificat **wildcard** `*.esig.tg` (DNS Challenge) est possible pour couvrir les 8 sous-domaines (et le sous-domaine d'administration, §10) en une fois.

7. Relais NOVA (assistant IA)

Le widget ne parle **jamais** directement à l'IA : il appelle un relais hébergé par l'ESIG, qui détient seul la clé API. Voir `shared/components/nova/README-NOVA.md`.

```
# 1. Copier data/formations.json dans la base documentaire du relais
cp data/formations.json shared/components/nova/base-documentaire/

# 2. Lancer le relais avec la clé API en variable d'environnement (jamais dans le code)
CLE_API_MODELE=sk-ant-xxxx \
ORIGINES AUTORISEES="https://esig.tg,https://admission.esig.tg,https://executive.esig.tg,https://cooperation.esig.tg,https://carrieres.esig.tg,https://alumni.esig.tg,https://news.esig.tg,https://tech.esig.tg" \
node shared/components/nova/serveur-relais.js
```

Le mettre en **service systemd** (redémarrage auto) sur le serveur B, écoutant sur `127.0.0.1:8787`. Il est exposé **par le Nginx du serveur B** en `/api/assistant`, **sur chaque sous-domaine** (bloc `location = /api/assistant` du §5) — donc en **même origine** que le widget. NPM transmet la requête au serveur B comme n'importe quelle page ; **aucune route API supplémentaire à créer dans NPM**.

Modèle par défaut : `claude-haiku-4-5-20251001` (modifiable via la variable `MODELE`).

Chaîne d'IP (limitation de débit). Le relais lit `X-Forwarded-For` pour retrouver l'IP réelle du visiteur. La chaîne est : visiteur → NPM → Nginx (B) → relais. NPM et Nginx B doivent **transmettre X-Forwarded-For** (NPM le fait par défaut ; côté B, `proxy_add_x_forwarded_for` au §5). Le relais prend la **première** IP de la liste (déjà géré dans `serveur-relais.js`).

Puis activer l'endpoint **relatif** dans les pages (§3.1).

CORS. L'appel étant same-origin, le CORS n'est plus sur le chemin critique. La liste `ORIGINES AUTORISEES` du relais reste utile comme défense (elle inclut déjà `tech.esig.tg`).

8. Formulaire — service de traitement (fourni)

Le site comporte **11 formulaires** : **préinscription**, **rendez-vous conseiller**, **demande d'information** (admission), **devis** (executive), **contact**, **demande de document officiel** (§9), **candidature internationale** et **devenir partenaire** (coopération), **dépôt d'offre** (carrières), **proposer un projet / partenariat** (tech), **rejoindre / mettre à jour un profil Alumni** (alumni). Ils sont tous branchés sur un **service de traitement fourni** : `shared/components/formulaires/serveur-formulaires.js` (mode d'emploi complet : `shared/components/formulaires/README-FORMULAIRES.md`).

Principe (identique au relais NOVA) : le client (`shared/js/formulaires.js`) envoie la demande en `POST /api/formulaire` (même origine) ; le service la transmet **par e-mail** au bon service selon le formulaire. Pot de miel anti-spam, limitation de débit (6/10 min/IP), consentement RGPD, **secrets (SMTP) en variables d'environnement**.

Acheminement e-mail par défaut (objet `DESTINATAIRES` , modifiable) : - `admissions@esig.tg` ← préinscription, demande d'info, rendez-vous, candidature internationale - `formation@esig.tg` ← devis - `contact@esig.tg` ← contact, demande de document, dépôt d'offre (carrières), **devenir partenaire (coopération), proposer un projet (tech), rejoindre le réseau Alumni**

Mise en service (serveur B) :

```
npm install nodemailer      # une fois (pour l'envoi e-mail)
SMTP_HOTE=smtp.esig.tg SMTP_PORT=587 SMTP_USER=site@esig.tg SMTP_MDP=***
EXPEDITEUR=site@esig.tg \
  node shared/components/formulaires/serveur-formulaires.js
```

- À placer en **service systemd**, écoutant sur `127.0.0.1:8788` , exposé via Nginx en `/api/formulaire` (bloc du §5). Endpoint des pages passé en **relatif** au déploiement (§3.2).
- La liste des origines autorisées du service inclut déjà les **8 sous-domaines** (dont `tech.esig.tg`) ; ajustable via la variable `ORIGINES_AUTORISEES` .
- **Sans SMTP configuré**, le service tourne quand même et enregistre les demandes dans `_boite-reception/` (repli, aucune demande perdue ; dossier `_*` exclu du déploiement — à sécuriser et purger).

RGPD : les données restent sur l'infrastructure ESIG (aucun tiers). Détails dans le README.

9. Documents officiels réservés (agrément, certificat)

Décision de la direction : **l'agrément d'État et le certificat de qualité ne sont PAS en téléchargement libre**. Ils sont remis **sur demande motivée, après autorisation**.

- Les deux PDF sont rangés dans `medias/_documents-reserves/` et **exclus automatiquement du déploiement** (`assembler.js` retire **tout dossier** `medias/_*` , ainsi que `_boite-reception`). **Ne jamais les recopier** dans un dossier public (`medias/documents/` , etc.).

- Le site propose une page « **Demander un document officiel** » (`demande-document.html` , `noindex`) : demande motivée → **vérification/autorisation** par l'établissement → **remise manuelle** du document.
- **Contrôle à faire après déploiement** : `https://esig.tg/medias/documents/agrement-esig.pdf` doit renvoyer **404** (le fichier ne doit pas être en ligne).

10. Espace d'administration du contenu (CMS)

Pour permettre à la Direction Marketing & Communication et à l'Administrateur général de **publier sans toucher au code** (actualités, agenda, médias), un **CMS léger auto-hébergé** est prévu (décision : option A).

Spécification complète : `docs/CMS-SPEC.md` ; modèle de données : `docs/CMS-MODELE-CONTENU.md` .

Points d'installation (équipe technique) : - Installer le CMS (**Directus** recommandé ; alternative PocketBase) sur le serveur B. - L'exposer via NPM sur un **sous-domaine privé** (ex. `admin.esig.tg` / `cms.esig.tg`), **SSL + accès restreint** (IP/VPN si possible). Le CMS n'est **pas** public au même titre que les 8 sites. - Brancher le script de synchronisation `build/sync-cms.js` (récupère le contenu → `data/*.json`), puis régénérer/redéployer (build + assembler). - **Comptes & rôles** (voir §14) : créer le super-administrateur, activer la **2FA**, **inviter** les personnes. **Chaque personne définit elle-même son mot de passe** (aucun mot de passe créé par un tiers).

11. En-têtes de sécurité — récapitulatif

`X-Content-Type-Options: nosniff` · `X-Frame-Options: DENY` · `Referrer-Policy: strict-origin-when-cross-origin` · `Permissions-Policy` restrictif · `Content-Security-Policy` (cf. §5). **Strict-Transport-Security (HSTS) est posé par NPM** (serveur A, avec le SSL). Les autres en-têtes se posent soit dans le Nginx du serveur B (§5), soit dans NPM (onglet *Advanced*) — **ne pas les dupliquer** aux deux endroits.

La CSP autorise **Cloudflare Web Analytics** : `script-src ... https://static.cloudflareinsights.com` et `connect-src 'self' https://cloudflareinsights.com` (§11 bis).

11 bis. Web analytics — Cloudflare Web Analytics (sans cookie, RGPD)

Solution retenue : **Cloudflare Web Analytics** — gratuite, **sans cookie**, sans bandeau de consentement supplémentaire, aucune donnée envoyée à un tiers publicitaire.

- Le **beacon** est déjà intégré au **pied de page partagé** (`shared/components/footer.html`) → présent sur les **8 sous-domaines** et les 206 pages. La **CSP** l'autorise déjà (§5).
- **Reste à faire** : Cloudflare → *Analytics & Logs* → *Web Analytics* → *Add a site* (`esig.tg`), copier le **token**, puis remplacer `CF_BEACON_TOKEN_A_REEMPLACER` dans `footer.html` (une seule fois) et relancer le déploiement (`deploiement/deploy.sh`).
- Le beacon est **retiré automatiquement de la préviz Netlify** (les statistiques ne concernent que la production). Détails : `deploiement/COMMANDES.md` (§5 bis).

12. Redirections 301 depuis l'ancien site

L'ancien site mono-page (hébergé temporairement sur `news.esig.tg`) doit rediriger vers les nouvelles pages. Exemples à placer dans le vhost concerné :

```
location = /formation-continue.html { return 301 https://executive.esig.tg/; }
location = /admission.html           { return 301 https://admission.esig.tg/admission.html; }
}
location = /actualites.html          { return 301 https://news.esig.tg/; }
location = /contact.html              { return 301 https://esig.tg/contact.html; }
location ^~ /formations/bts/          { return 301 https://admission.esig.tg$request_uri; }
location ^~ /formations/licence/      { return 301 https://admission.esig.tg$request_uri; }
location ^~ /formations/master/       { return 301 https://admission.esig.tg$request_uri; }
location ^~ /formations/continue/     { return 301 https://executive.esig.tg$request_uri; }
location ^~ /formations/modulaire/    { return 301 https://executive.esig.tg$request_uri; }
location ^~ /formations/langues/      { return 301 https://executive.esig.tg$request_uri; }
```

12.1 — Anciens sous-domaines renommés (transition)

« International » devient **Coopération** et « Entreprises » est absorbé par **Carrières**. Conserver le DNS de `international.esig.tg` et `entreprises.esig.tg` le temps de la transition, et créer dans NPM (serveur A) deux hôtes qui **redirigent (301)** tout leur trafic vers les nouveaux espaces :

```
server { server_name international.esig.tg; return 301
https://cooperation.esig.tg$request_uri; }
server { server_name entreprises.esig.tg; return 301
https://carrieres.esig.tg$request_uri; }
```

(SSL de ces deux hôtes également géré par NPM.)

13. Tests post-déploiement (liste de contrôle)

- ☐ Les **8 sous-domaines** répondent en **HTTPS** (cadenas valide, redirection HTTP→HTTPS).
 - ☐ Page d'accueil de chaque site : logo, menu, **bandeau écosystème**, pied de page, widget WhatsApp OK.
 - ☐ **admission** : recherche des 66 formations, ouverture d'une **fiche**, formulaire de préinscription.
 - ☐ **executive** : recherche des 79 formations, page Langues, formulaire de **devis**.
 - ☐ **news** : magazine + ouverture d'un **article** (URL `?slug=...`) + page **Agenda** (7 événements).
 - ☐ **tech** : galerie **Innovations** (filtres recherche/domaine/statut), ouverture d'une **fiche projet** (`?slug=...`), formulaire « **Proposer un projet** ».
 - ☐ **cooperation** : Partenaires, Mobilité, page **Projets**, formulaire « **Devenir partenaire** ».
 - ☐ **carrieres** : page **Offres** (filtre Stage / Emploi / Alternance), **Espace recruteurs** (dépôt d'offre).
 - ☐ **alumni** : **Portraits**, formulaire « **Rejoindre le réseau** ».
 - ☐ **Redirections** : `international.esig.tg` → `cooperation` , `entreprises.esig.tg` → `carrieres` (§12.1).
 - ☐ Liens du **bandeau écosystème** (8 espaces) entre sous-domaines : aucun 404 (double par `verifier-liens.js`).
 - ☐ **Documents réservés** : `.../medias/documents/agrement-esig.pdf` renvoie **404** ; la page « Demander un document » s'affiche.
 - ☐ `https://<site>/sitemap.xml` et `/robots.txt` accessibles sur chaque sous-domaine.
 - ☐ En-têtes de sécurité présents (outils du navigateur ou `securityheaders.com`).
 - ☐ **NOVA** : si activé, une question obtient une réponse ; sinon le widget est masqué (sur les 8 sites).
 - ☐ **Formulaires** : si le traitement (§8) est branché, un envoi test arrive bien par e-mail (tester au moins un formulaire par destinataire : `admissions@` , `formation@` , `contact@`).
 - ☐ Validation **W3C** (`validator.w3.org`) et **Lighthouse** sur les pages d'accueil (objectif ≥ 90).
 - ☐ Redirections 301 depuis les anciennes URLs testées.
-

14. Rôles & accès

Détail complet : `docs/ADMINISTRATION-ROLES.md` . En résumé, deux couches : - **Contenu** (CMS, §10) : Administrateur général (super-admin) + Administrateur éditorial (DMC). - **Infrastructure** (serveur, NPM, SSL, sauvegardes) : Admin technique (Digital Hub) + **Admin réseau senior (M. Kalipé)**. Ce sont des **accès serveur/NPM**, pas des comptes du site.

Règles : **moindre privilège**, **2FA** partout, **aucun mot de passe créé par un tiers** (invitation + mot de passe défini par chacun), journalisation des actions sensibles.

15. Mettre à jour le site plus tard

- **Contenu formations** : éditer la source, relancer `convertir-formations.js` + `generer-fiches.js` (admission + executive) + `build.js` + `assembler.js` + `verifier-liens.js` .
 - **Actualités** : éditer `data/actualites.json` (voir `sites/news/COMMENT-AJOUTER-UNE-ACTU.md`), puis `assembler.js` .
 - **Agenda** : éditer `data/agenda.json` (voir `sites/news/COMMENT-AJOUTER-UN-EVENEMENT.md`), puis `assembler.js` .
 - **Contenu ESIG TECH** (sans code) : éditer `data/projets-tech.json` , `actualites-tech.json` , `emissions-tech.json` (**y remplacer les liens YouTube par les vraies URL des vidéos**), `evenements-tech.json` , `laboratoires-tech.json` , `partenaires-tech.json` , puis `assembler.js` .
 - **Pages** : éditer les `.src.html` , relancer `build.js` + `assembler.js` + `verifier-liens.js` .
 - **Via le CMS** (une fois en place, §10) : la publication régénère les données et le site.
-

16. Annexe — documents de référence

Fichier	Objet
docs/CMS-SPEC.md	Spécification de l'espace d'administration (CMS)
docs/CMS-MODELE-CONTENU.md	Modèle de données du CMS (collections & champs)
docs/ADMINISTRATION-ROLES.md	Les profils et la matrice de droits
docs/api-spec.md	Contrat de l'API du relais NOVA
shared/components/nova/README-NOVA.md	Fonctionnement et sécurité du relais NOVA
shared/components/formulaires/README-FORMULAIRES.md	Service de traitement des formulaires
docs/CHANGELOG.md	Historique des versions (dont v2.2.0 — réorganisation en 8 espaces)
medias/_documents-reserves/LISEZ-MOI.md	Rappel : documents réservés, ne pas exposer

Écosystème ESIG Global Success — 8 sites, 206 pages. Site statique sécurisé, relais IA à clé protégée, formulaires en même origine, SSL et HSTS via NPM. Bon déploiement.